SORTING ALGORITHMS

Implementation, Timing & Comparison

Selection Sort • Insertion Sort • Bubble Sort • Merge Sort

Programming Language	Java
Array Size	1,000 random elements
Number of Runs	5 runs per algorithm; average time recorded
IDE	Eclipse

Prepared as a practical assignment and rough-copy reference.

1. Objective

The objective of this experiment is to implement four sorting algorithms in Java using an array of 1,000 random numbers. Their average execution times are measured and compared. Their time complexity, space complexity, advantages, disadvantages, and suitable use cases are also studied.

2. Experimental Method

• Generate one array containing 1,000 random integers.

• Give each sorting algorithm a separate copy of the same original array so the comparison is fair.

• Run each algorithm five times and calculate its average execution time.

• Measure sorting time using Java's `System.nanoTime()`. Convert nanoseconds to milliseconds.

• Check that each result is sorted correctly.

3. Experimental Results

The following values are taken from the user's Eclipse output. Timings can vary between runs and computers.

Algorithm	Array Size	Average Time (ms)	Rank
Selection Sort	1000	1.8731	3rd
Insertion Sort	1000	1.2333	2nd
Bubble Sort	1000	3.3423	4th
Merge Sort	1000	0.3297	1st

Finding: Merge Sort was fastest in this experiment (0.3297 ms). Bubble Sort was slowest (3.3423 ms). These are measured results for this run environment, not guaranteed timings for every computer.

4. Algorithm 1 — Selection Sort

Definition: Selection Sort repeatedly finds the smallest element in the unsorted part and swaps it into the next correct position.

Algorithm

Step 1: Start
Step 2: Read array A of n elements
Step 3: Set i = 0
Step 4: Set min = i
Step 5: Set j = i + 1
Step 6: If A[j] < A[min], then set min = j
Step 7: Increase j by 1. If j < n, go to Step 6
Step 8: Swap A[i] and A[min]
Step 9: Increase i by 1. If i < n - 1, go to Step 4
Step 10: Stop

Pseudocode

```
SelectionSort(A, n)
for i = 0 to n - 2
min = i
for j = i + 1 to n - 1
if A[j] < A[min]
min = j
swap A[i] and A[min]
End
```

Complexity

Case	Time Complexity

Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$

Space Complexity: O(1), because it uses constant extra memory.

Advantages: Easy to understand. Uses very little memory. Makes few swaps.

Disadvantages: Slow for big arrays. Always makes many comparisons. Usually not stable.

Best scenario: Small arrays or cases where minimizing swaps is important.

5. Algorithm 2 — Insertion Sort

Definition: Insertion Sort takes one element at a time and inserts it into the correct position in the sorted part of the array.

Algorithm

Step 1: Start
Step 2: Read array A of n elements
Step 3: Set i = 1
Step 4: Set key = A[i] and j = i - 1
Step 5: While j >= 0 and A[j] > key, set A[j + 1] = A[j] and j = j - 1
Step 6: Set A[j + 1] = key
Step 7: Increase i by 1. If i < n, go to Step 4
Step 8: Stop

Pseudocode

```
InsertionSort(A, n)
for i = 1 to n - 1
key = A[i]
j = i - 1
while j >= 0 and A[j] > key
A[j + 1] = A[j]
j = j - 1
A[j + 1] = key
End
```

Complexity

Case	Time Complexity
Best	O(n)
Average	$O(n^2)$
Worst	$O(n^2)$

Space Complexity: O(1).

Advantages: Simple to write. Fast for small arrays. Fast for nearly sorted data. Stable.

Disadvantages: Slow for big unsorted arrays. Shifts many elements.

Best scenario: Small or nearly sorted arrays, or data that arrives gradually.

6. Algorithm 3 — Bubble Sort

Definition: Bubble Sort compares adjacent elements and swaps them if they are in the wrong order.

Algorithm

Step 1: Start
Step 2: Read array A of n elements
Step 3: Set i = 0
Step 4: Set swapped = false and j = 0
Step 5: If A[j] > A[j + 1], swap them and set swapped = true
Step 6: Increase j by 1. If j <= n - 2 - i, go to Step 5
Step 7: If swapped = false, go to Step 9
Step 8: Increase i by 1. If i <= n - 2, go to Step 4
Step 9: Stop

Pseudocode

```
BubbleSort(A, n)
for i = 0 to n - 2
swapped = false
for j = 0 to n - 2 - i
if A[j] > A[j + 1]
swap A[j] and A[j + 1]
swapped = true
if swapped == false
break
End
```

Complexity

Case	Time Complexity
Best (optimized)	O(n)
Average	O(n²)
Worst	O(n²)

Space Complexity: O(1).

Advantages: Very easy to understand. Uses very little memory. Stops early if data is already sorted.

Disadvantages: Slow for big arrays. Makes many comparisons and swaps.

Best scenario: Learning sorting concepts and sorting very small arrays.

7. Algorithm 4 — Merge Sort

Definition: Merge Sort divides the array into smaller parts, sorts them, and merges them back in order. It uses divide and conquer.

Algorithm

Step 1: Start
Step 2: Read array A, set left = 0 and right = n - 1
Step 3: If left >= right, go to Step 8
Step 4: Set mid = left + (right - left) / 2
Step 5: Call MergeSort(A, left, mid)
Step 6: Call MergeSort(A, mid + 1, right)
Step 7: Merge the two sorted halves
Step 8: Stop

Pseudocode

```
MergeSort(A, left, right)
if left < right
mid = left + (right - left) / 2
MergeSort(A, left, mid)
MergeSort(A, mid + 1, right)
Merge(A, left, mid, right)
End
```

Merge procedure

Step 1: Create temporary arrays L and R for the two halves
Step 2: Set i = 0, j = 0 and k = left
Step 3: If L[i] <= R[j], copy L[i] to A[k] and increase i. Else copy R[j] and increase j
Step 4: Increase k by 1. Repeat Step 3 until L or R is finished
Step 5: Copy the remaining elements of L or R into A

Complexity

Case	Time Complexity
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

Space Complexity: O(n), because temporary arrays are used.

Advantages: Fast for big data. Speed is always the same. Stable.

Disadvantages: Needs extra memory. Harder to write. Slower on very tiny arrays.

Best scenario: Large datasets when reliable performance and stable sorting are important.

8. Overall Complexity Comparison

Algorithm	Best	Average	Worst	Extra Space
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Bubble Sort*	O(n)	O(n²)	O(n²)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)

*Bubble Sort best case is O(n) in the optimized version with a swap flag, as used in this program.

9. Which Algorithm Should Be Used?

Scenario	Recommended Algorithm	Reason
Small array	Insertion Sort	Simple and efficient for small inputs.
Nearly sorted data	Insertion Sort	Needs few shifts.
Minimize swaps	Selection Sort	At most n – 1 swaps.
Learning basics	Bubble Sort	Easy to understand.
Large dataset	Merge Sort	O(n log n) time.
Stable sorting needed	Merge Sort	Preserves order of equal elements.

10. Conclusion

In this experiment, four sorting algorithms were tested on 1,000 random elements, with five runs per algorithm. Merge Sort was the fastest in the recorded results at 0.3297 ms, while Bubble Sort was the slowest at 3.3423 ms. Merge Sort is generally suitable for large datasets, while Insertion Sort can be effective for small or nearly sorted arrays.

Actual timings may vary by computer and runtime conditions.

Note: The measured times are specific to this experiment. Complexity describes how performance grows as input size increases.